

Production-Grade Resume Validation + ATS

Decision Logic for ResiLens AI

You are correct.

Right now your system still feels partially hardcoded because:

- every uploaded file still enters scoring flow
- invalid documents still receive AI analysis
- score thresholds alone decide outcomes
- there is no recruiter-grade validation pipeline

A real-world ATS first determines:

1. Is this actually a resume?
2. Does it contain professional structure?
3. Is there meaningful experience?
4. Does it match the JD?
5. Is the candidate worth reviewing?

Only AFTER validation should scoring begin.

WHAT YOU SHOULD CHANGE

Inside `main.py`

You should completely replace your current scoring/result logic with the following architecture.

STEP 1 — ADD THIS FUNCTION

Place this BELOW:

```
calculate_match_score()
```

ADD:

```
def validate_resume_structure(resume_text: str):  
  
    if not resume_text:
```

```

        return False, "Empty document"

cleaned_text = resume_text.lower().strip()

if len(cleaned_text) < 120:
    return False, "Document too small to be a resume"

resume_keywords = {
    "experience",
    "education",
    "skills",
    "projects",
    "internship",
    "work",
    "linkedin",
    "github",
    "email",
    "phone",
    "objective",
    "summary",
    "certification",
    "achievements",
}

matched_keywords = [
    keyword
    for keyword in resume_keywords
    if keyword in cleaned_text
]

keyword_score = len(matched_keywords)

has_email = bool(
    re.search(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
        cleaned_text,
    )
)

has_phone = bool(
    re.search(
        r"\b\d{10}\b",
        cleaned_text,
    )
)

resume_sections = 0

```

```

for section in [
    "education",
    "skills",
    "experience",
    "projects",
]:
    if section in cleaned_text:
        resume_sections += 1

if keyword_score < 3:
    return False, "Not enough resume keywords found"

if resume_sections < 2:
    return False, "Resume sections missing"

if not has_email:
    return False, "No email detected"

if not has_phone:
    return False, "No phone number detected"

return True, "Valid resume"

```

STEP 2 — INSIDE /analyze ENDPOINT

FIND:

```
resume_text = extract_resume_text(
```

AFTER extraction ADD:

```

    is_valid_resume, validation_reason = (
        validate_resume_structure(
            resume_text
        )
    )

    if not is_valid_resume:

        result = {

            "fileName": file.filename,

```

```

        "result": "Rejected",

        "confidence": 0,

        "skillsMatch": 0,

        "experienceFit": 0,

        "analysisMode": analysis_mode,

        "jobDescription": job_description,

        "time": datetime.now().strftime(
            "%d %b %Y, %I:%M %p"
        ),

        "preview": resume_text[:2000],

        "feedback": (
            "The uploaded file does not appear "
            "to be a valid professional resume.\n\n"
            f"Validation Reason: {validation_reason}"
        ),

        "matchedSkills": [],

        "resumePath": saved_file_path,
    }

    inserted = collection.insert_one(
        result
    )

    result["_id"] = str(
        inserted.inserted_id
    )

    return result

```

STEP 3 — REPLACE YOUR CURRENT RESULT LOGIC

REMOVE:

```
if final_score >= 70:
```

AND REPLACE WITH:

```
    matched_skills_count = len(
        matched_skills
    )

    resume_length = len(
        resume_text.split()
    )

    if (
        final_score >= 78
        and matched_skills_count >= 5
        and resume_length >= 250
    ):

        final_result = "Shortlisted"

    elif (
        final_score >= 45
        and matched_skills_count >= 2
        and resume_length >= 140
    ):

        final_result = "Needs Review"

    else:

        final_result = "Rejected"
```

STEP 4 — MAKE AI FEEDBACK REALISTIC

Inside:

```
get_ai_feedback()
```

UPDATE PROMPT STYLE TO:

```

prompt = f"""
You are a senior recruiter.

IMPORTANT:
- Never assume the candidate is good.
- Be strict and realistic.
- Reject weak resumes.
- Mention exact missing skills.
- Mention technical weaknesses.
- Mention if the resume is low quality.
- Mention if candidate lacks alignment.
- Think like a real ATS + recruiter.

Resume Match Score: {score}%
Final Result: {final_result}

Job Description:
{jd_text}

Resume:
{resume_text[:3000]}
"""

```

WHAT THIS FIXES

NOW:

Random PDFs

- Rejected
- 0% score
- invalid resume reason

DSA Notes

- Rejected
- no fake AI feedback
- no fake 50% match

Tiny resumes

- rejected

Weak resumes

- rejected realistically

Medium resumes

- needs review

Strong resumes

- shortlisted
-

WHY THIS WILL FEEL LIKE A REAL PRODUCT

Because now your platform has:

- validation layer
- recruiter logic
- structural resume analysis
- intelligent rejection
- realistic ATS pipeline
- professional scoring behavior

This is how actual hiring platforms behave.

Without this, users immediately feel:

"This is hardcoded."

With this:

"This actually behaves like an ATS."

IMPORTANT FINAL IMPROVEMENT

LATER you should also add:

- semantic embeddings
- experience extraction
- role-specific scoring
- skill weighting
- JD semantic similarity

- anti-keyword-spam detection
- education relevance
- years-of-experience extraction

THAT is what will eventually make ResiLens AI launch-ready as a real SaaS platform.